

National University of Computer and Emerging Sciences
Karachi Campus

Compiler Construction (CS-4031)

Sessional-I Exam

Date: 02/25/2026

Total Time: 1 hr

Course Instructors

Total Marks: 30

Shaharbano Asif, Muhammad Kashif, Zulfiqar Ali, Ghulam Murtaza

Total Questions: 03

Roll No

Section

Student Signature

Do not write below this line.

Attempt all questions.

CLO # 1: Understand the basic techniques used in compiler construction such as lexical analysis, top-down, bottom-up parsing, context-sensitive analysis, and intermediate code generation.

Question 1: Provide brief answers to the following:

(2 x 5 = 10 points)

- a. List down the inputs of each phase of compiler.

Phase	Input
1. Lexical Analysis (Scanning)	Source Code
2. Syntax Analysis (Parsing)	Tokens
3. Semantic Analysis	Parse Tree / AST
4. Intermediate Code Generation	Annotated AST / Symbol Table
5. Code Optimization	Intermediate Code (IC)
6. Code Generation	Optimized Intermediate Code
7. Symbol Table Management	Identifiers and Attributes

- b. Differentiate between lexeme and token?

Answer: A **token** is a category or a type of lexical unit (like "Keyword" or "Identifier"), while a **lexeme** is the actual string of characters from the source code that matches that category (like "int" or "myVar")

- c. Discuss the role that a symbol table play during lexical analysis.

Answer: The symbol table's role during lexical analysis is to **store identifiers** and **differentiate them from keywords** by checking against pre-loaded entries. It ensures that when the same lexeme appears again, it is recognized as the same identifier.

- d. Why does a compiler construction need NFA and DFA?

Answer: A compiler needs NFA and DFA for **lexical analysis** to recognize tokens using regular expressions. **NFAs** help describe patterns easily, while **DFAs** are efficient for implementation since they require minimal backtracking.

- e. Classify the compiler phases into front end and back end components

Answer: **Front End** phases (analysis) include Lexical Analysis, Syntax Analysis, and Semantic Analysis,

which are language-dependent and produce intermediate code. **Back End** phases (synthesis) include Code Optimization and Code Generation, which are machine-dependent and produce target code.

CLO # 2: Understand the basic data structures used in compiler construction such as abstract syntax trees, symbol tables, three-address code, and stack machines.

Question 2:

(5 x 2 = 10 points)

- a) For the give C code, list all tokens and categorize them e.g., keywords, identifiers, operators (arithmetic, relational, logical, etc.), constants and punctuations.

```
int main() {  
    int a = 10, b = 20;  
    float 2value = 5.5;  
    char ch = 'AB';  
    int $total = 100;  
    int num@ = 5;  
    a = a + b * 2;  
    return 0; }
```

Lexeme	Token Type	Category
int	Keyword	Data Type
main	Identifier	Function Name
(	Punctuation	Parenthesis
)	Punctuation	Parenthesis
{	Punctuation	Brace (Opening)
int	Keyword	Data Type
a	Identifier	Variable Name
=	Operator	Assignment
10	Constant	Integer Constant
,	Punctuation	Comma
b	Identifier	Variable Name
=	Operator	Assignment
20	Constant	Integer Constant
;	Punctuation	Semicolon
float	Keyword	Data Type
2value	Invalid Identifier	(Starts with digit)
=	Operator	Assignment
5.5	Constant	Float Constant
;	Punctuation	Semicolon
char	Keyword	Data Type
ch	Identifier	Variable Name
=	Operator	Assignment
'AB'	Invalid Constant	(Multi-character constant)
;	Punctuation	Semicolon
int	Keyword	Data Type
\$total	Invalid Identifier	(Special char not allowed)
=	Operator	Assignment
100	Constant	Integer Constant
;	Punctuation	Semicolon

int	Keyword	Data Type
num@	Invalid Identifier	(Special char not allowed)
=	Operator	Assignment
5	Constant	Integer Constant
;	Punctuation	Semicolon
a	Identifier	Variable Name
=	Operator	Assignment
a	Identifier	Variable Name
+	Operator	Arithmetic (Addition)
b	Identifier	Variable Name
*	Operator	Arithmetic (Multiplication)
2	Constant	Integer Constant
;	Punctuation	Semicolon
return	Keyword	Return Statement
0	Constant	Integer Constant
;	Punctuation	Semicolon
}	Punctuation	Brace (Closing)

- b) Consider the code given in Part a, fill the information in the symbol table only specific to Lexical Analyzer.

Answer: A Lexical Analyzer **does not** determine the following:

- **Data Type** (int, float, char) — This is determined by the **Semantic Analyzer**.
- **Scope** — This is determined by the **Syntax/Semantic Analyzer**.
- **Kind** (variable/function/array) — This is determined later.

Token Name	Lexeme	Token Type	Data Type	Scope	Declaration Line	Kind
1	main	Identifier (Function)			1	
2	a	Identifier			2	
3	b	Identifier			2	
4	2value	Invalid Identifier	Error		3	Error
5	ch	Identifier			4	
6	'AB'	Invalid Char Constant	Error		4	Error
7	\$total	Invalid Identifier	Error		5	Error
8	num@	Invalid Identifier	Error		6	Error

CLO # 3: Design and implement a compiler using a software engineering approach.

Question 3:

(4 + 6 = 10 points)

- a) A lexer designer defines the following rules for a small language:
Keywords (int), Identifier ([a-z][a-z0-9]*), For the input string: **intx= -10;**

Explain and mention the techniques/algorithms how the lexer resolves conflicts between

- Keyword **int** vs **identifier** starting with **int**
- Unsigned integer 10 vs signed integer -10

Answer:

1. Keyword int vs Identifier starting with int

- **Technique: Maximal Munch (Longest Match) Rule**
- **Explanation:** The lexer scans characters left-to-right and takes the longest possible sequence that matches a rule. For `intx`, it reads `i→n→t` (matches keyword `int`), but the next character `x` is still valid for identifier `[a-z0-9]*`. Since `intx` as a whole matches **Identifier** rule, the lexer takes `intx` as a single **Identifier** token, not keyword `int` followed by identifier `x`.

2. Unsigned Integer 10 vs Signed Integer -10

- **Technique: Treating - as Separate Operator**
- **Explanation:** The lexer does **not** recognize `-10` as a single signed integer token. Instead, it treats `-` as a separate **operator/minus** token and `10` as an **unsigned integer constant** token. The combination is later handled by the parser during syntax/semantic analysis.

(b) Draw combined NFA for the following.

- Identifier: starts with a letter or `_`, followed by letters, digits, or `_`.
- Numbers both Signed/Unsigned 0-9
- Relational operators: `<`, `>`, `<=`, `>=`, `==`, `!=`